

AI Advent Challenge #8

Неделя 3. Лекция 3

Memory, состояние и stateful-агенты

Ключевая мысль: Главная идея лекции: хороший агент - это не просто LLM, которая отвечает на сообщения. Это управляемая система: профиль пользователя + состояние задачи + разрешённые переходы + инварианты + валидация ответа.

Основа файла: сырая транскрибация лекции, конспект другого участника, summary и скриншоты презентации. Текст очищен от мусора распознавания, терминология исправлена, добавлены технические уточнения, практические выводы и структура для выполнения задания недели.

Содержание

- 1. Контекст недели и главный результат
- 2. Stateless vs stateful agents
- 3. Персонализация: один request - разные response
- 4. Где хранить профиль и как подмешивать его в prompt
- 5. Task State Machine
- 6. Сохранение и возобновление задачи
- 7. Инварианты и валидация ответа
- 8. Антипаттерны
- 9. Архитектура stateful-агента
- 10. Практическое задание недели
- 11. Корректировки, замечания и что было дополнено
- 12. Чек-листы для реализации и видео

1. Контекст недели и главный результат

Третья неделя продолжает тему второй недели: сначала разобрались с context management, теперь нужно понять memory и state. Это следующий шаг к созданию собственных агентов: не просто посылать разрозненные сообщения в LLM, а строить помощника, который знает пользователя, понимает текущую задачу, не перепрыгивает этапы и соблюдает ограничения.

В лекции memory рассматривается не только как short-term и long-term memory, а шире - как часть архитектуры stateful-агента. Такой агент можно встроить в кодовый ассистент, Telegram-бота, сервис доставки, помощника для обучения, внутренний корпоративный ассистент, генератор мобильных приложений и т.д.

Ключевая мысль: Цель недели - собрать из персонализации, state machine, инвариантов и prompt builder полноценный stateful-agent flow.

Что должно получиться к концу недели

- агент хранит профиль пользователя и настройки проекта;
- агент понимает текущий этап задачи: planning, execution, validation или done;
- агент не нарушает порядок выполнения задачи;
- агент проверяет response по инвариантам;
- агент умеет сохранить состояние и продолжить после паузы;
- архитектура не привязана жёстко к конкретной модели: OpenAI, DeepSeek, GigaChat и т.д. должны быть заменяемыми реализациями LLM engine.

2. Stateless vs stateful agents

2.1. Stateless agent

Stateless-агент отвечает на каждый запрос почти с нуля. Даже если история сообщений сохранена в файле или внутри текущей сессии, у него может не быть долговременного профиля пользователя, правил приложения, state machine и инвариантов.

Пример из лекции: пользователь просит: 'Напиши сервис авторизации'. Если агент не знает, что проект всегда пишется на Kotlin/Ktor, он может выдать типовое решение на Python или другом наиболее вероятном стеке. Это не потому, что Python всегда правильный, а потому что без профиля и ограничений модель выбирает наиболее распространённую и статистически вероятную траекторию ответа.

Признаки stateless-подхода

- каждый запрос фактически начинается почти с нуля;
- нет долговременного профиля пользователя или проекта;
- глобальный контекст задачи легко теряется после новой сессии или summary;
- нет правил и ограничений: стек, город доставки, запрещённые технологии, бюджет;
- агент перепрыгивает этапы: делает то, что пользователь попросил прямо сейчас;
- нет долгосрочной стратегии и веса процесса;
- разные пользователи получают примерно одинаковые ответы, без настоящей персонализации.

Уточнение / корректировка: В сырой транскрибации есть место, где сказано 'Особенности Stateful...', но дальше перечисляются признаки stateless-агента: запрос идёт с нуля, нет долговременной истории, агент забывает контекст. В итоговом конспекте это исправлено.

2.2. Stateful agent

Stateful-агент работает не только с текущим prompt, но и с состоянием. Он знает, на каком этапе находится задача, какой профиль у пользователя, какие ограничения действуют и какие переходы между этапами разрешены.

Пример: состояние говорит, что задача - 'планирование сервиса авторизации', стек - Kotlin + Ktor, порядок работы - сначала план, потом код. Если пользователь пишет: 'Пропусти план, сразу код', агент не должен соглашаться автоматически. Он должен проверить state machine и сказать, что сначала нужно утвердить план.

Stateful-агент должен:

- помнить этап задачи;
- хранить контекст в слоях;
- знать ограничения проекта и пользователя;
- контролировать transitions;
- адаптировать ответ под профиль;
- сохранять состояние для продолжения работы после паузы.

Ключевая мысль: Stateful здесь не означает, что сама LLM магически получила память. State живёт во внешнем слое: файлах, базе данных, state storage, profile storage, prompt builder и validation logic.

2.3. Сравнение

Критерий	Stateless agent	Stateful agent
Память	Локальная история или вообще без памяти	Профиль, состояние задачи, summary, сохранённые результаты
Процесс	Prompt -> response	Task -> state -> allowed transition -> response -> validation
Персонализация	Минимальная или отсутствует	Стиль, constraints, контекст пользователя
Ограничения	Легко забываются	Вставляются в prompt и проверяются кодом
Риск	Галлюцинации, перепрыгивание этапов	Меньше хаоса, но нужна архитектура

3. Персонализация: один request - разные response

Первый строительный блок stateful-агента - персонализация. Смысл: один и тот же запрос должен давать разные ответы в зависимости от пользователя, его цели, стиля и доменных ограничений.

3.1. Три группы данных профиля

Блок профиля	Что содержит	Пример
Style	Формат, тон, глубина ответа	Кратко/подробно, формально/разговорно, с кодом/без кода
Constraints	Ограничения проекта и пользователя	Python 3.12, FastAPI, без Django, только бесплатные API
Context	Зачем пользователь пришёл и какой результат нужен	Готовится к интервью, делает Telegram-бота, учит язык для поездки

3.2. Интервью при первом запуске

Практичный способ собрать профиль - инициализационное интервью. Например, Telegram-бот видит новый `user_id` и сначала спрашивает стиль, ограничения и цель. После этого сохраняет профиль в отдельный файл или базу.

Пример вопросов для стартового интервью:

1. В каком стиле отвечать: кратко, подробно, с примерами кода?
2. Какой стек использовать по умолчанию?
3. Какие технологии запрещены?
4. Какая главная цель: учёба, проект, интервью, рабочая автоматизация?
5. Что считать хорошим результатом?

3.3. Пример вне программирования

Если агент помогает учить иностранный язык, профиль должен различать цели: TOEFL/IELTS, разговорная практика, путешествие, работа, чтение документации. При одном и том же запросе 'объясни past perfect' ответ для экзамена и ответ для путешествия должны быть разными.

4. Где хранить профиль и как подмешивать его в prompt

Слайды презентации показывают, что известные инструменты используют похожую архитектуру, но разные места хранения и score. Важно не копировать один вариант blindly, а выбирать способ хранения под задачу.

Инструмент	Где хранится	Score	Что задаёт	Как инжектится
Claude	User Preferences + Project Instructions	Глобальный / per-project / per-style	Тон, формат, роль, контекст	Автоматически в system prompt
Cursor	.cursor/rules/ и .md-файлы	Per-project, per-file pattern	Стек, архитектура, code standards	Через правила проекта и контекст редактора
Claude Code	CLAUDE.md в корне repo	Per-repo, наследуется	Стек, команды, агенты, правила	Prepend к каждому вызову агента
ChatGPT	Custom Instructions + Memory	Глобальный / per-GPT	Роль, предпочтения, контекст	Автоматически в system prompt

Уточнение / корректировка: Скриншот презентации был использован как источник для таблицы. В Markdown-конспекте другого участника эта таблица описана общими словами, но конкретное сравнение Claude/Cursor/Claude Code/ChatGPT было полезно добавить.

4.1. Общий pipeline персонализации

Архитектура в общем виде одна: profile storage -> profile loader -> prompt builder -> LLM -> personalized response.

```
profile = load_profile(user_id)
prompt = build_prompt(query=user_query, profile=profile)
response = llm.generate(prompt)
```

4.2. Что класть в system prompt

При создании собственного агента нужно явно решить, какие блоки попадут в prompt. В готовых инструментах часть этого скрыта, но если агент работает через HTTP-запрос к LLM, prompt assembly нужно проектировать самому.

- базовый system prompt: роль агента и общий режим работы;
- profile: стиль, constraints, контекст;
- summary предыдущей сессии, если оно действительно нужно;
- текущий task state;

- инварианты и запрещённые решения;
- текущий user query.

Уточнение / корректировка: Не нужно бездумно класть всё в каждый prompt. Если профиль содержит правила для разных режимов - коддинг, изучение языка, документы - подмешивай только релевантный слой.

5. Task State Machine

Персонализации недостаточно. Агент должен работать не как свободный чат, а как управляемый процесс. Для этого нужна task state machine - конечный автомат задачи.

5.1. Базовые стадии

State	Смысл	Типичные артефакты
PLANNING	Сбор требований и утверждение плана	requirements, план, список вопросов
EXECUTION	Создание решения	код, текст, артефакты, изменения файлов
VALIDATION	Проверка результата	тесты, ревью, сравнение с планом
DONE	Фиксация финального результата	итоговый ответ, сохранённый результат, обновление state

На слайде презентации показана именно такая цепочка: PLANNING -> EXECUTION -> VALIDATION -> DONE. Лектор подчёркивает, что стадии можно расширять, но убирать базовые этапы стоит крайне осторожно.

5.2. Allowed transitions

Allowed transitions дают детерминизм недетерминированному агенту. Модель может сгенерировать что угодно, но приложение не обязано принимать любой переход.

```
from enum import Enum

class TaskState(str, Enum):
    PLANNING = "planning"
    EXECUTION = "execution"
    VALIDATION = "validation"
    DONE = "done"

ALLOWED_TRANSITIONS = {
    TaskState.PLANNING: {TaskState.EXECUTION},
    TaskState.EXECUTION: {TaskState.VALIDATION, TaskState.PLANNING},
    TaskState.VALIDATION: {TaskState.DONE, TaskState.EXECUTION},
    TaskState.DONE: set(),
}

def transition(current: TaskState, target: TaskState) -> TaskState:
    if target not in ALLOWED_TRANSITIONS[current]:
        raise ValueError(f"Transition {current.value} -> {target.value} is forbidden")
    return target
```

5.3. Почему одних промптов мало

Если просто написать transitions в prompt, это уже поможет. Но текстовые правила могут потеряться после summary/compacting, быть вытеснены новым контекстом или нарушены моделью. Для железных ограничений transitions лучше хранить и проверять в коде.

Ключевая мысль: Prompt - это инструкция. Code validation - это ограничитель. Хороший агент использует оба слоя.

5.4. Injecting state

State можно вставлять в prompt через prompt builder. На слайде 'Injecting State' показан блок, где в prompt добавляются STATE, CURRENT, PLAN, DONE, PROFILE и QUERY, а правила говорят: работай только в рамках current step, не перепрыгивай этапы, если step завершён - верни next_step.

```
def build_prompt(query: str, task: dict, profile: dict) -> str:
    return f'''
[STATE]
{task['state']} step {task['step']}/{task['total_steps']}

[CURRENT]
{task['current']}

[PLAN]
{task['plan']}

[DONE]
{task.get('done', '')}

[PROFILE]
{profile}

[QUERY]
{query}

Rules:
- Work only inside the current step.
- Do not jump to future stages.
- If the step is complete, return next_step.
'''
```

6. Сохранение и возобновление задачи

Stateful-агент должен уметь продолжать работу после паузы. Пропал интернет, закрыли компьютер, закончились токены, прошло 12 часов - агент должен загрузить state и продолжить с нужного места.

Что сохранять

- task_id и user_id;
- текущий state: planning/execution/validation/done;
- утверждённый план;
- результаты каждого этапа;
- profile пользователя;
- constraints и invariants;
- последний user query и важные решения;
- список allowed transitions или версия state machine.

```
{
  "task_id": "auth-service-001",
  "user_id": "victor",
  "state": "execution",
  "plan": ["define API", "implement JWT", "add tests"],
  "current": "implement JWT token validation",
  "done": ["requirements collected", "plan approved"],
  "profile_ref": "profiles/victor.yaml",
  "invariants_ref": "projects/ai_advent/invariants.yaml"
}
```

Уточнение / корректировка: Не стоит хранить только summary. Summary удобно для компактности, но оно теряет детали. Структурный state лучше хранить отдельно: stage, plan, constraints, results.

7. Инварианты и валидация ответа

Инварианты - это ограничения, которые не должны меняться от запроса к запросу. В лекции они часто описаны как 'то, что запрещено', но технически инвариант может быть и запретом, и обязательным требованием.

7.1. Примеры инвариантов

Тип	Примеры
Стек	Python 3.12, FastAPI, PostgreSQL, Redis; или Kotlin, Ktor, Compose
Архитектура	MVI, ViewModel для состояния, слои domain/application/infrastructure
Запреты	не использовать RxJava, AsyncTask, Java, платные API
Доменные ограничения	доставка только по Москве; только японский язык; только бесплатные модели
Процесс	сначала plan, потом execution, потом validation

7.2. Injecting invariants

На слайде 'Injecting Invariants' показан pipeline: Invariants -> Prompt Builder -> LLM -> Validate -> Pass/Fail. То есть инварианты сначала вставляются в prompt, затем response проверяется отдельно.

```
class Invariant:
    def __init__(self, name: str, description: str, forbidden_patterns: list[str]):
        self.name = name
        self.description = description
        self.forbidden_patterns = forbidden_patterns

    def check(self, response: str) -> list[str]:
        violations = []
        lower = response.lower()
        for pattern in self.forbidden_patterns:
            if pattern.lower() in lower:
                violations.append(f"Invariant '{self.name}' violated: found '{pattern}'")
        return violations

def validate_response(response: str, invariants: list[Invariant]) -> list[str]:
    errors = []
    for invariant in invariants:
        errors.extend(invariant.check(response))
    return errors
```

7.3. Retry loop

```
while True:
    query = read_user_query()

    prompt = build_prompt(
        query=query,
        profile=profile,
        state=state,
        invariants=invariants,
    )

    response = llm.generate(prompt)
    violations = validate_response(response, invariants)

    if not violations:
        send(response)
        state = next_state(state, response)
        save_state(state)
        break

    query = "Исправь ответ. Нарушения: " + "; ".join(violations)
```


Ключевая мысль: Инварианты - это как линтер для требований, которые выражены человеческим языком, а не только синтаксисом кода.

8. Антипаттерны

Антипаттерн	Почему плохо	Как исправить
Всё в одном prompt	Контекст переполняется, модель теряет фокус, ответ деградирует	Делить контекст на слои и подмешивать только нужное
Нет validation transitions	LLM может услужливо согласиться на любой переход	Transitions проверять в коде: map + require/raise
Инварианты только в тексте	Prompt - просьба, а не жёсткий запрет	Дублировать: prompt + code validation + warning/retry
Summary вместо state	Summary теряет детали и может исказить смысл	Хранить структурный state отдельно
Один профиль на всё	Профиль для кодинга мешает профилю для обучения языку	Scope профиля: global/per-project/per-task/per-mode

9. Архитектура stateful-агента

Слайд с архитектурой показывает простой цикл: User Query + Profile + State Machine + Invariants -> Prompt Builder -> LLM -> Validate -> Pass/Fail. Если Pass - response отдаётся пользователю и state обновляется. Если Fail - retry с описанием нарушений.

9.1. Компоненты

Компонент	Ответственность
ProfileStorage	Хранит и загружает профиль пользователя: style, constraints, context
TaskStateStorage	Хранит состояние задачи, план, текущий этап и результаты
StateMachine	Проверяет разрешённые transitions
InvariantChecker	Проверяет response по правилам и запретам
PromptBuilder	Собирает system prompt / user prompt из релевантных блоков
LLMEngine	Абстракция над провайдером модели: OpenAI, DeepSeek, GigaChat, локальная LLM
ResponseValidator	Объединяет invariant check, формат ответа, проверку state output

9.2. Почему важны интерфейсы

Не нужно писать код так, чтобы весь агент зависел от конкретного OpenAI/DeepSeek-клиента. Лучше определить интерфейс LLMEngine и инжектировать конкретного провайдера. Тогда можно менять модель, не переписывая prompt builder, state machine и validation.

```

from typing import Protocol

class LLMEngine(Protocol):
    def generate(self, prompt: str) -> str:
        ...

class PromptBuilder(Protocol):
    def build(self, query: str, profile: dict, state: dict, invariants: list[dict]) -> str:
        ...

class Agent:
    def __init__(self, llm: LLMEngine, prompt_builder: PromptBuilder):
        self.llm = llm
        self.prompt_builder = prompt_builder

```

Ключевая мысль: Даже 2-3 хорошо спроектированных файла с state machine, prompt builder и invariant checker могут резко улучшить работу агента или код-ассистента.

10. Практическое задание недели

По смыслу лекции итог недели - реализовать Stateful Agent. Задания недели, вероятно, будут идти по нарастающей: персонализация, state machine, инварианты, затем объединение в общий pipeline.

10.1. Рекомендуемый проект для сдачи

Оптимальный формат для тебя: небольшой Python CLI/Telegram-like mock, который демонстрирует stateful flow без лишней инфраструктуры.

```

ai-advent-week3-stateful-agent/
├── README.md
├── pyproject.toml
├── .env.example
├── app/
│   ├── main.py
│   ├── agent.py
│   ├── llm_engine.py
│   ├── prompt_builder.py
│   ├── profile_storage.py
│   ├── task_state.py
│   ├── invariants.py
│   └── validators.py
├── data/
│   ├── profiles/
│   │   └── victor.yaml
│   ├── tasks/
│   │   └── auth_service.json
│   └── invariants/
│       └── default.yaml
└── tests/
    ├── test_state_machine.py
    ├── test_invariants.py
    └── test_prompt_builder.py

```

10.2. Минимальный MVP

- CLI принимает user_id и query;
- если профиля нет - проводит короткое интервью;
- профиль сохраняется в YAML/JSON/MD;
- task имеет state: PLANNING, EXECUTION, VALIDATION, DONE;
- allowed transitions проверяются кодом;
- инварианты подмешиваются в prompt и отдельно проверяют response;
- при нарушении invariant агент делает retry или возвращает ошибку;
- state сохраняется на диск и может быть загружен после перезапуска.

10.3. Демо-сценарий для видео

- Показать профиль: стиль ответа, стек, ограничения.
- Создать задачу: 'сервис авторизации'.
- Показать, что агент сначала входит в PLANNING и не пишет код сразу.
- Попросить: 'пропусти план, сразу код' - агент должен отказать или попросить утвердить план.
- Утвердить план - перейти в EXECUTION.
- Смоделировать response с запрещённой технологией, например RxJava/Java/Django.
- Показать, что invariant checker ловит нарушение и просит переделать.
- Перезапустить приложение и показать, что task state восстановился.

10.4. Что написать в README

Раздел	Что указать
Purpose	Демонстрация stateful-agent flow: profile + state machine + invariants + validation
Architecture	Компоненты: profile storage, state machine, prompt builder, LLM engine, validator
How to run	Команды запуска и переменные окружения
Demo cases	3-4 сценария: персонализация, запрет transition, invariant fail/pass, resume
Limitations	Это учебный playground, не production-agent

11. Корректировки, замечания и что было дополнено

11.1. Что исправлено в транскрибации

- 'копины' -> Kotlin; 'кейтер' -> Ktor; 'стоит машина' -> state machine; 'стейф-легенда' -> stateful agent;
- 'ЛМК/ЛМП' -> LLM; 'респонс/реквест' оставлены как response/request, но в нормальной форме;
- блок 'Особенности Stateful...' по смыслу описывал stateless-агента - исправлено;
- 'саморепредыдущего' -> summary предыдущей сессии;
- 'всё в одном фронте' -> всё в одном prompt;
- сомнительное 'орен-слай' не закреплялось как факт: в итоговом тексте смысл сформулирован без привязки к неясно распознанному названию.

11.2. Технические уточнения

- LLM сама по себе не становится stateful. Stateful-поведение создаётся внешним слоем: storage, prompt builder, state machine, validation.
- System prompt важен, но не абсолютен. Если запрет критичный, его нужно проверять кодом.
- Инвариант - это не только запрет, но и обязательное свойство: 'использовать FastAPI', 'ответ в JSON', 'сначала план'.
- State machine не гарантирует истинность ответа модели. Она гарантирует порядок процесса и снижает хаос.
- Summary нельзя считать полноценным state storage. Summary помогает сжать историю, но структурные данные нужно хранить отдельно.
- Персонализация должна быть scoped: global/per-project/per-task/per-mode. Один огромный профиль на всё ухудшает качество.

11.3. Замечания к файлам другого участника

Файлы 03-memory-state-notes.md и 03-memory-state-summary.md в целом корректные: основная логика лекции сохранена, структура понятная, ключевые блоки stateful/stateless, персонализация, state machine, инварианты и архитектура описаны верно.

Что было дополнено в этой версии: конкретная таблица по Claude/Cursor/Claude Code/ChatGPT из скриншотов презентации; реконструкция слайдов про Profile Pipeline, Transitions, Injecting Invariants, Injecting State и Anti-patterns; практический Python-oriented MVP; отдельный блок корректировок и технических уточнений.

12. Чек-листы для реализации и видео

12.1. Implementation checklist

- [] Есть ProfileStorage: профиль можно создать, загрузить и обновить.
- [] Есть TaskState enum/class: PLANNING, EXECUTION, VALIDATION, DONE.
- [] Есть ALLOWED_TRANSITIONS и тесты на запрещённые переходы.
- [] Есть PromptBuilder, который собирает query + profile + state + invariants.
- [] Есть InvariantChecker и минимум 2-3 инварианта.
- [] Есть retry/feedback при нарушении invariant.
- [] State сохраняется и восстанавливается после перезапуска.
- [] LLM provider спрятан за интерфейсом LLMEngine.
- [] README объясняет архитектуру и показывает demo commands.

12.2. Video checklist

- [] Показать структуру проекта.
- [] Показать профиль и инварианты.
- [] Запустить агента.
- [] Показать различие ответа с профилем и без профиля.
- [] Показать запрещённый transition.
- [] Показать invariant violation и исправление.
- [] Показать сохранение и восстановление state.
- [] Коротко проговорить: это не просто prompt, а stateful-agent pipeline.

12.3. Короткий итог

Ключевая мысль: Хороший stateful-агент появляется не из одного длинного промпта. Он появляется из системы: profile + memory layers + task state machine + invariants + prompt builder + validation + persistence.

Источники, использованные для подготовки

- Nueva grabacion 1m4a.txt - сырая транскрибация лекции 3.
- 03-memory-state-notes.md - подробный конспект другого участника.
- 03-memory-state-summary.md - краткое summary.
- 1.pdf - скриншоты презентации, 4 страницы: таблица инструментов, pipeline профиля, state machine, transitions, injecting invariants/state, архитектура и антипаттерны.

Файл подготовлен как учебный конспект: дословность транскрибации не сохранялась там, где распознавание искажало термины или мешало смыслу. Приоритет был на техническую точность и практическую применимость для выполнения задания недели.